

Design and Synthesis of Full Adder using Yosys

INTRODUCTION:

In this lab, students will learn to synthesize a Full Adder circuit using Yosys. The design takes three inputs (A, B, and Cin) and produces Sum and Carry outputs. Students will understand how a Full Adder extends the functionality of a Half Adder by handling carry input, making it suitable for building multi-bit binary adders used in ALUs, calculators, and processors.

APPLICATION:

Full Adders are used for binary addition in ALUs, processors, and calculators. By cascading them, multi-bit adders like ripple carry and parallel adders are built, making them essential for arithmetic and digital signal processing tasks.

PROCEDURE TO CREATE A LAB:

Step 1: Set up Project Directory

- 1.1: Create a directory for your lab: `mkdir <project_name>`
- 1.2: Open lab directory: `cd <Project_name>`
- 1.3: Create directory for project: `mkdir fulladder`
- 1.4: Open project directory: `cd fulladder`

Step 2: Create Design file

- 2.1: Open gvim editor for design with command: `gvim full_adder.v`

RTL Design Code:

```
module full_adder(  
    input a, b, cin,  
    output sum, cout);  
  
    assign sum = a ^ b ^ cin;  
    assign cout = (a & b) | (a & cin) | (b & cin);  
  
endmodule
```

- 2.2: Save the file using command- `:wq!`

Step 3: Synthesize the Design with Yosys

invoke Yosys in a new terminal by the following command
`yosys`

3.1: Use the following command to Synthesize:

```
# Read the Verilog source file
read_verilog full_adder.v

# Set the top-level module and check hierarchy
hierarchy -check -top full_adder

# high level stuff
proc; opt; memory; opt;

# mapping to internal cell library
techmap; opt

# mapping logic to standard cell library
abc -liberty osu018_stdcells.lib

# Clean up unused cells and signals after mapping
clean

# Write the final synthesized gate-level netlist to a new Verilog file
write_verilog full_adder_synth.v

#Generate a graphical visualization of the synthesized circuit
show
```

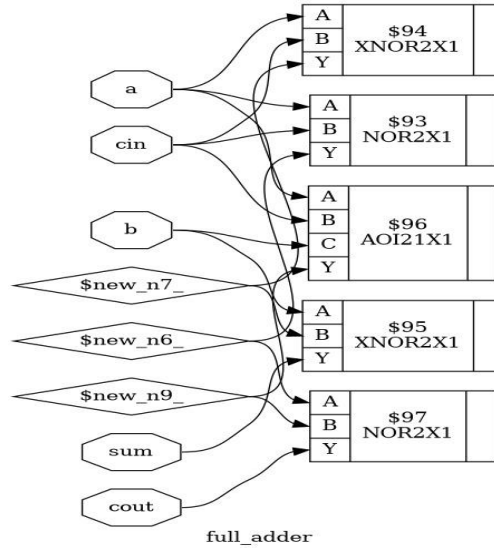
Step 4: View Output File (Netlist)

```
gvim full_adder_synth.v
```

Observation :

- The Full Adder design is synthesized without errors.
- Combinational logic gates (AND, OR, XOR) are used to implement the sum and carry outputs.
- The sum output correctly reflects the binary addition of the input bits and carry-in.
- The carry-out logic is accurately generated based on input combinations, enabling proper chaining for multi-bit addition.

Output :



Conclusion:

The Yosys synthesis output confirms the correct implementation of the Full Adder design. The RTL description is successfully read, optimized, and mapped to the target standard cell library. The generated gate-level netlist accurately reflects the intended combinational logic, with proper use of logic gates to compute the sum and carry outputs based on the three input bits (A, B, and Cin). The synthesis report verifies that the logic functions are correctly implemented, and the design hierarchy and connectivity match the expected behavior of a Full Adder. This ensures that the hardware can reliably perform single-bit binary addition and can be cascaded to construct multi-bit adders for use in more complex arithmetic operations.